RELATÓRIO TÉCNICO DE AUDITORIA DE CÓDIGO-FONTE

Relatório de Auditoria de Segurança — PsicoConnect

Avaliação Técnica Sistemática de Vulnerabilidades em 5 Categorias Críticas

---

Data da Auditoria:	15 de Setembro de 2026
Projeto Avaliado:	PsicoConnect (Versão 2.0.0 - Monorepo)
Escopo Auditado:	backend/ (src/server.js, routes/, middleware/, config/), database/ (schema.sql, seed.sql), frontend/ (HTMLs, js/script.js), .github/workflows/ci.yml
Metodologia:	Análise Estática de Código (SAST) orientada a OWASP Top 10 e modelagem de ameaças multitenancy
Classificação:	**CONFIDENCIAL — RESTRIÇÃO DE ACESSO**

Nota Metodológica & Stack Detectada:

O ecossistema do **PsicoConnect** foi inspecionado em sua totalidade. Foi detectada a seguinte stack técnica:
• **Linguagem & Runtime:** JavaScript / Node.js (ES Modules, v22+)
• **Backend Framework:** Express 4.21.2 com middlewares de rate limiting e helmet
• **Banco de Dados & Acesso:** PostgreSQL nativo via driver pg (Pool connection), sem ORM; queries executadas com raw SQL parametrizado
• **Autenticação & Sessão:** JWT (JSON Web Tokens via jsonwebtoken) + bcryptjs para hashing de senha
• **Frontend:** Arquitetura híbrida (Vanilla HTML5 + JS DOM manipulation + CSS3) com persistência client-side simulada em localStorage e chamadas parciais REST para a API
• **Deploy & CI/CD:** GitHub Actions (.github/workflows/ci.yml) com pipeline de testes PostgreSQL e build para Render.

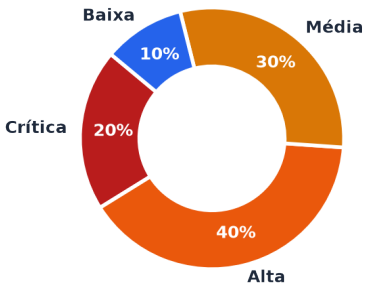
Mapeamento das 5 Categorias:
1. *Banco sem Tranca:* PostgreSQL sem RLS configurada em schema.sql, demandando isolamento lógico por filtros WHERE usuario_id = $x no Express.
2. *Permissão no Navegador:* Checagem de discrepâncias entre restrições visuais do frontend (tipo 'Administrador') vs. validação com requireRole('admin') nas rotas Express.
3. *IDOR:* Inspeção de parâmetros de rota (:id, :agendamentoId) sem cruzamento de propriedade com req.user.id.
4. *Chaves Expostas:* Inspeção de fallbacks permissivos em variáveis de ambiente, credenciais fixas de demonstração em código e scripts SQL.
5. *Inputs sem Tratamento (XSS):* Auditoria de manipulação de DOM via innerHTML sem bibliotecas de sanitização (como DOMPurify) no frontend.

Resumo Executivo

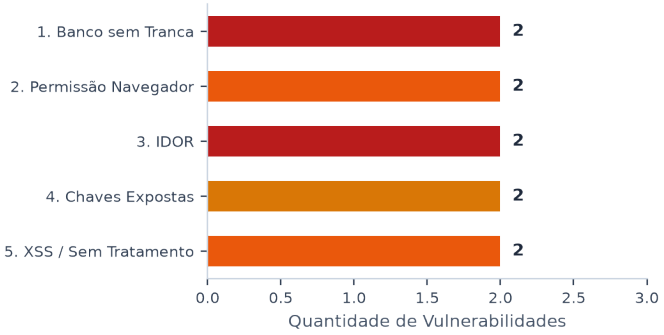
A auditoria de segurança identificou um total de **10 achados de segurança confirmados no código**, distribuídos em **2 Críticos**, **4 Altos**, **3 Médios** e **1 Baixo**. Adicionalmente, foram mapeados **4 pontos fortes defensivos** que demonstram boas práticas parciais de engenharia de software (como hashing de senhas com bcrypt, queries SQL parametrizadas prevenindo SQL Injection clássico, e rate limiting estruturado contra ataques de força bruta).

2	4	3	1	4
Críticas	Altas	Médias	Baixas	Pontos Fortes

Achados por Severidade



Vulnerabilidades por Categoria Auditada



Pontos Fortes e Riscos Centrais

PONTOS FORTES IDENTIFICADOS (EVIDÊNCIAS)	RISCOS CENTRAIS DA ARQUITETURA
1. Prevenção de SQL Injection: No backend (databaseRoutes.js e authRoutes.js), todas as consultas SQL utilizam queries parametrizadas ($1, $2...) do driver pg, sem interpolação direta de strings em comandos SQL.	**1. Vazamento Horizontal de Dados Clínicos (IDOR):** Falta de validação de propriedade em rotas de mensagens e cancelamento de consultas permite que qualquer paciente ou psicólogo autenticado acesse conversas confidenciais de terceiros e cancele agendamentos alheios.
2. Armazenamento de Senhas: Hashing seguro implementado com bcrypt.hash(senha, 10) e conferência com bcrypt.compare em authRoutes.js:46, 121.	**2. Fragilidade no Segredo JWT:** A existência de fallback com segredo estático público em authMiddleware.js:4 possibilita forjar tokens com perfil admin se a variável de ambiente for omitida em produção.
3. Rate Limiting Estruturado: Em server.js:36-50, aplicação de express-rate-limit global (100 reqs/15m) e limite restrito para autenticação (20 reqs/15m), mitigando brute-force.	**3. Exposição a XSS no Frontend:** O uso generalizado de innerHTML sem sanitização em script.js permite sequestro de sessão e manipulação de prontuários clínicos locais.
4. Proteção em Rotas Administrativas: A rota de validação de psicólogos (databaseRoutes.js:55) valida adequadamente autenticação e privilégio via middleware requireAuth, requireRole('admin').	**4. Rota Crítica de Agendamento do Meet Aberta:** POST /api/meet/create-event permite geração irrestrita de reuniões e envio de spam via Google Calendar API.

Tabela de Achados Detalhados por Categoria

Sev.	Cat.	Arquivo:Linha	Descrição do Problema & Impacto
ALTA	1. Banco sem Tranca	backend/src/routes/databaseRoutes.js:207-221	**GET /api/mensagens/:agendamentoId sem filtro de participante:** A consulta SQL não valida se req.user.id é o paciente ou psicólogo vinculado ao agendamento. Qualquer usuário logado lê mensagens privadas de qualquer consulta.
ALTA	1. Banco sem Tranca	backend/src/routes/databaseRoutes.js:224-241	**POST /api/mensagens sem validação de tenant/participação:** Insere mensagens no chat usando apenas agendamento_id vindo do body, permitindo injeção de mensagens fraudulentas em sessões alheias.
MÉDIA	2. Permissão Navegador	frontend/js/script.js:155, 209-215	**Bypass de tela de Admin no Frontend via credencial mock:** Validação de acesso ao painel de aprovação restrita a session()?.tipo === 'Administrador' e execução client-side de aprovarPsicologo() sobre localStorage.
MÉDIA	2. Permissão Navegador	backend/src/routes/databaseRoutes.js:78-108	**POST /api/psicologos público e sem autenticação:** Permite criação direta de registros de psicólogo sem senha e sem passar pelo fluxo de autenticação e validação de perfil formal.
CRÍTICA	3. IDOR	backend/src/routes/databaseRoutes.js:189-204	**PATCH /api/agendamentos/:id/cancelar sem verificação de posse:** Executa UPDATE agendamentos SET status = 'cancelado' WHERE id = $1 diretamente. Qualquer usuário autenticado pode cancelar consultas de outros pacientes/psicólogos conhecendo apenas o UUID.
ALTA	3. IDOR	backend/src/server.js:136-168	**POST /api/meet/create-event desprotegido:** Endpoint público sem requireAuth que aceita qualquer consultaId e e-mails de destino, gerando reuniões e convidando e-mails arbitrários via API real do Google Calendar.
CRÍTICA	4. Chaves Expostas	backend/src/middleware/authMiddleware.js:4	**Segredo JWT com fallback padrão hardcoded:** Se JWT_SECRET não for configurada no ambiente (como no Render ou local), a aplicação usa 'psicoconnect_dev_secret_troque_em_producao', permitindo forja irrestrita de tokens de admin.
BAIXA	4. Chaves Expostas	database/seed.sql:11-15 e script.js:152	**Credenciais Administrativas Padrão em Seed e Frontend:** Usuário admin@psicoconnect.com / admin@demo.com com senha 123456 inserido no banco de dados e exposto na interface de login de demonstração.
ALTA	5. Inputs (XSS)	frontend/js/script.js:247	**Cross-Site Scripting (XSS) Armazenado no Chat:** Mensagens digitadas pelo usuário são renderizadas via innerHTML sem sanitização ou escape HTML em byId('chatMensagens').innerHTML.
MÉDIA	5. Inputs (XSS)	frontend/js/script.js:238	**DOM-based XSS na Triagem de Sintomas:** O texto informado no campo de triagem (relatoTriagem) é utilizado para compor resposta HTML injetada via innerHTML em byId('resultadoTriagem') sem escape.

Recomendações Priorizadas de Correção

P1 (Imediato / Bloqueador de Release)

• **Falha no Segredo JWT (authMiddleware.js:4):** Remover imediatamente o valor de fallback. Implementar validação obrigatória no startup (if (!process.env.JWT_SECRET) throw new Error(...)) para abortar a inicialização caso a variável esteja ausente.

• **IDOR no Cancelamento de Agendamentos (databaseRoutes.js:189):** Modificar a cláusula UPDATE para verificar se id = $1 AND (paciente_id IN (SELECT id FROM pacientes WHERE usuario_id = $2) OR psicologo_id IN (SELECT id FROM psicologos WHERE usuario_id = $2)), garantindo que apenas participantes ou administradores cancelem a consulta.

P2 (Alta Prioridade / Próxima Sprint)

• **Isolamento de Mensagens do Chat (databaseRoutes.js:207, 224):** Incluir verificação de autorização em GET /api/mensagens/:agendamentoId e POST /api/mensagens, checando se o usuário autenticado pertence ao agendamento alvo antes de buscar ou persistir mensagens.

• **Autenticação e Validação no Meet (server.js:136):** Adicionar middleware requireAuth na rota /api/meet/create-event e validar se o usuário autenticado possui permissão para gerar evento associado àquela consulta.

• **Mitigação de XSS no Frontend (script.js:238, 247):** Substituir o uso de innerHTML por textContent na exibição de mensagens de chat e triagem, ou integrar biblioteca de sanitização robusta como DOMPurify antes de qualquer injeção de HTML.

P3 (Média / Baixa Prioridade / Melhorias Arquiteturais)

• **Depreciação de Rotas Redundantes (databaseRoutes.js:78):** Descontinuar a rota pública POST /api/psicologos em prol exclusivo do fluxo protegido em /api/auth/register.

• **Saneamento de Credenciais Padrão (seed.sql, script.js):** Assegurar que seeds de desenvolvimento não sejam executados em ambientes de homologação/produção e desabilitar credenciais demo em builds voltadas à produção.

• **Content Security Policy (CSP):** Reativar a diretiva CSP no middleware helmet em server.js:17-20 com políticas estritas de script-src e connect-src.

Issues Prontas para o GitHub

Blocos completos formatados em Markdown para criação imediata no repositório:

ISSUE #1 PARA O GITHUB: [Segurança] Vulnerabilidade Crítica de IDOR em Cancelamento de Agendamentos

````
--- ISSUE 1 ---
**Título:** [Segurança] Vulnerabilidade Crítica de IDOR em Cancelamento de Agendamentos
**Labels sugeridas:** security, severidade:crítica, p1

**Descrição do Problema:**
O endpoint PATCH /api/agendamentos/:id/cancelar recebe o identificador do agendamento via parâmetro de
rota e executa diretamente uma instrução UPDATE no banco de dados sem verificar se o usuário autenticado
(req.user.id) é o paciente ou psicólogo daquela consulta.

**Evidência de Código:**
backend/src/routes/databaseRoutes.js:189-204
```js
router.patch('/agendamentos/:id/cancelar', requireAuth, async (req, res) => {
const { id } = req.params;
const result = await query(
`UPDATE agendamentos SET status = 'cancelado' WHERE id = $1 RETURNING *`,
[id]
);
```

**Impacto de Segurança:**
Qualquer paciente ou psicólogo autenticado no sistema pode cancelar agendamentos de terceiros
arbitrários conhecendo ou iterando identificadores UUID.

**Sugestão de Correção:**
Adicionar verificação de pertinência antes da atualização ou restringir o UPDATE garantindo que a
consulta pertença ao usuário logado ou que req.user.tipo === 'admin'.

**Critérios de Aceite:**
- [ ] Usuário paciente não consegue cancelar agendamento de outro paciente (retorna 403).
- [ ] Usuário psicólogo não consegue cancelar agendamento onde não seja o profissional responsável
(retorna 403).
- [ ] Administrador consegue cancelar qualquer consulta.
- [ ] Teste automatizado de regressão cobrindo rejeição de IDOR implementado em database.test.js.
--- FIM ISSUE 1 ---
````

ISSUE #2 PARA O GITHUB: [Segurança] Segredo JWT com Fallback Público Hardcoded em authMiddleware.js

````
--- ISSUE 2 ---
**Título:** [Segurança] Segredo JWT com Fallback Público Hardcoded em authMiddleware.js
**Labels sugeridas:** security, severidade:crítica, p1

**Descrição do Problema:**
A constante JWT_SECRET utiliza um fallback estático caso a variável de ambiente process.env.JWT_SECRET
não seja fornecida, permitindo que a aplicação suba em produção com um segredo público conhecido.

**Evidência de Código:**
backend/src/middleware/authMiddleware.js:4
```js
const JWT_SECRET = process.env.JWT_SECRET || 'psicoconnect_dev_secret_troque_em_producao';
```

**Impacto de Segurança:**
Qualquer atacante externo pode gerar tokens JWT válidos com payload de administrador ({ id: 1, email:
'admin@attack.com', tipo: 'admin' }) e obter acesso irrestrito às rotas administrativas da API.

**Sugestão de Correção:**
Remover o valor padrão e incluir validação estrita no boot da aplicação (server.js / authMiddleware.js)
lançando exceção fatal se JWT_SECRET não estiver definida ou tiver menos de 32 caracteres.

**Critérios de Aceite:**
- [ ] Servidor aborta inicialização em ambiente não-teste se JWT_SECRET estiver indefinida.
- [ ] Fallback hardcoded removido de authMiddleware.js.
- [ ] .env.example e documentação atualizados destacando a obrigatoriedade da variável.
--- FIM ISSUE 2 ---
````

ISSUE #3 PARA O GITHUB: [Segurança] Vazamento de Dados do Chat e Injeção de Mensagens por Falta de Isolamento de Agendamento

````
--- ISSUE 3 ---
**Título:** [Segurança] Vazamento de Dados do Chat e Injeção de Mensagens por Falta de Isolamento de
Agendamento
**Labels sugeridas:** security, severidade:alta, p2

**Descrição do Problema:**
Os endpoints GET /api/mensagens/:agendamentoId e POST /api/mensagens não validam se o usuário
requisitante é participante da consulta relacionada ao agendamento informado.

**Evidência de Código:**
backend/src/routes/databaseRoutes.js:207-241
```js
router.get('/mensagens/:agendamentoId', requireAuth, async (req, res) => {
const result = await query(`SELECT m.*, u.nome AS autor_nome FROM mensagens m WHERE m.agendamento_id =
$1 ...`, [req.params.agendamentoId]);
```

**Impacto de Segurança:**
Quebra de sigilo profissional psicológico e privacidade clínica (LGPD / Conselho Federal de Psicologia).
Qualquer usuário logado pode ler todo o histórico de conversas entre médico e paciente de qualquer
consulta.

**Sugestão de Correção:**
Validar na tabela agendamentos se o agendamento_id pertence ao req.user.id antes de listar ou inserir
mensagens.

**Critérios de Aceite:**
- [ ] GET /api/mensagens/:id rejeita com 403 requisições de usuários que não participam da consulta.
- [ ] POST /api/mensagens rejeita com 403 tentativa de postagem de terceiros.
- [ ] Testes de integração cobrindo os cenários autorizados e negados.
--- FIM ISSUE 3 ---
````

ISSUE #4 PARA O GITHUB: [Segurança] Endpoint de Criação de Eventos Meet Desprotegido contra Abuso e IDOR

````
--- ISSUE 4 ---
**Título:** [Segurança] Endpoint de Criação de Eventos Meet Desprotegido contra Abuso e IDOR
**Labels sugeridas:** security, severidade:alta, p2

**Descrição do Problema:**
A rota POST /api/meet/create-event não possui middleware de autenticação (requireAuth) e aceita
livremente parâmetros arbitrários de consultaId, pacienteEmail e psicologoEmail.

**Evidência de Código:**
backend/src/server.js:136-168
```js
app.post('/api/meet/create-event', async (req, res) => {
const { consultaId, summary, description, startDateTime, endDateTime, pacienteEmail, psicologoEmail } =
req.body;
```

**Impacto de Segurança:**
Possibilidade de exaustão de cota da API Google Calendar, envio de e-mails de phishing/spam utilizando a
credencial da clínica e criação não autorizada de links de videoconferência.

**Sugestão de Correção:**
Adicionar requireAuth e validar se o consultaId corresponde a um agendamento válido pertencente ao
usuário logado.

**Critérios de Aceite:**
- [ ] Rota rejeita requisições anônimas com status 401.
- [ ] Usuário não autorizado para a consulta recebe 403.
- [ ] Dados de e-mail dos participantes são extraídos do banco de dados a partir do agendamento, não do
corpo da requisição.
--- FIM ISSUE 4 ---
````

ISSUE #5 PARA O GITHUB: [Segurança] Vulnerabilidade de Cross-Site Scripting (XSS) no Chat e Triagem

````
--- ISSUE 5 ---
**Título:** [Segurança] Vulnerabilidade de Cross-Site Scripting (XSS) no Chat e Triagem
**Labels sugeridas:** security, severidade:alta, p2

**Descrição do Problema:**
No frontend, as mensagens do chat e os resultados de triagem de sintomas utilizam innerHTML concatenando
variáveis providas por usuários sem qualquer sanitização.

**Evidência de Código:**
frontend/js/script.js:238 e 247
```js
// Chat:
byId('chatMensagens').innerHTML = banco.mensagens.map(m => `<div
class="chat-bubble">...<p>${m.texto}</p></div>`);
// Triagem:
byId('resultadoTriagem').innerHTML = `<div class="notice-box"><b>Sugestão:</b> ${sugestao}...`;
```

**Impacto de Segurança:**
Execução de scripts maliciosos no contexto de execução do navegador de outros usuários e psicólogos,
possibilitando roubo de tokens JWT em localStorage e sequestro de sessão.

**Sugestão de Correção:**
Criar elementos DOM com document.createElement e atribuir textos via textContent, ou integrar a
biblioteca DOMPurify para sanitizar strings antes de injetar em innerHTML.

**Critérios de Aceite:**
- [ ] Inserção de carga <img src=x onerror=alert(1)> no chat renderiza como texto puro sem disparar
script.
- [ ] Inserção de payload no relato de triagem é escapada com segurança.
- [ ] Diretiva CSP ativa no backend proibindo scripts inline inseguros.
--- FIM ISSUE 5 ---
````